UNIVERSIDAD NACIONAL DE LOJA

Desarrollo de un módulo para la Gestión de inventario en una embotelladora de Agua (RainTamia Plus).

Nombre: Isaac David Ramon Tinoco

Asignatura: Software Engineering Models

Fecha: 05/02/2026

Año Lectivo

2025-2026

1. Introducción.

Contexto y Problemática

El presente proyecto de software está dirigido para la PYME embotelladora de agua "RainTamia Plus". La gestión actual del inventario en la empresa presenta deficiencias críticas en lo que respecta al control físico, la planificación de reabastecimiento, y la trazabilidad en perdidas, afectando esto de manera directa en lo que respecta a la eficiencia operativa y la rentabilidad del propio negocio, resultando en la incapacidad de satisfacer pedidos, esto último pudiendo ser por falta de insumos necesarios para poder elaborar los productos, o también, existencia de insumos necesarios, pero que estos no se hayan utilizado para elaborar productos.

Problemas identificados

- **Falta de control de reabastecimiento**: La dependencia de métodos manuales para el control de existencias de materia prima (envases, tapas, sellos, etiquetas) resulta en un manejo ineficiente de la gestión de mínimos y máximos. No existe un mecanismo automático que pueda generar una solicitud de orden de compra cuando el stock de insumos cae por debajo del punto de reorden.
- **Opacidad en la trazabilidad de pérdidas:** Existe una incapacidad para mantener un historial destallado de movimientos como "daño" o "perdida", especialmente los que ocurren durante la distribución. La falta de un registro formal de daño/faltante en la entrega imposibilita la aplicación contable de la responsabilidad al transportista, impactando directamente en el valor final del inventario.

Propuesta de valor

Se propone un módulo para la gestión del inventario en la embotelladora "RainTamia Plus", el cual será la fuente de la verdad sobre el stock de materia prima y productos terminados. Esta centralizara el control para los insumos como envases de plástico, tapas y etiquetas.

1. Automatización de reabastecimiento: El módulo centralizara el control tanto de mínimos y máximos, disparando de manera automática solicitudes de orden de compra al dueño para la aprobación de estas.
2. Control físico y lógico: Permitirá el registro detallado de todos los movimientos realizados (entradas, ajustes, salidas).
3. Integridad de costos: Mantendrá actualizado el costo unitario de cada ítem y permitirá la valoración monetaria del stock, asegurando la precisión de los datos que consume el área financiera.

Propósito

El propósito de este proyecto es desarrollar un módulo de gestión de inventarios para la PYME embotelladora de agua "RainTamia Plus", usando el enfoque de Arquitectura Dirigida por Modelos (MDA). Este enfoque permite abstraer la lógica de negocio de los detalles técnicos, garantizando que el sistema responda fielmente a a las necesidades de control de stock y trazabilidad de la empresa, facilitando su mantenimiento y evolución tecnológica.

Descripción

El proyecto se desarrolla bajo la metodología de MDA, estructurándose en 3 niveles de abstracción:

- CIM. – Corresponde a modelar los procesos de negocio de la embotelladora, identificando cuellos de botella en el reabastecimiento y la perdida de insumos.
- PIM. – Se definen los patrones y el estilo arquitectónico que se usaran, y resuelven la logica de inventarios sin ligarse a una tecnología en específico.
- PSM. – Corresponde a la transformación del diseño independiente de la plataforma, al uso de tecnologías concretas, para este proyecto, las tecnologías son Django, Svelte y SQLite3.

Funcionalidades Cubiertas:

- Automatización de alertas por movimientos del inventario y bajo stock (RF5, RF6, RF7).
- Registro de movimientos (entrada, salida, ajuste, daño) de la materia prima y productos (RF7).
- Gestión de materia prima y productos (RF7, RF12).
- Guardado y restauración de estados del inventario (RF10, RF11).

Funcionalidades No Cubiertas:

- Generación de solicitudes de orden de compra.
- Flujo de aprobación/rechazo de ordenes de compra para el propietario.
- Valoración monetaria del stock y control de costos unitarios/gastos totales por insumo.
- Gestión de unidades agrupadas.

Público Objetivo:

- Estudiantes y desarrolladores. – Como un caso de estudio sobre la aplicación de MDA, tanto para tomar las decisiones correctas aplicadas en el proyecto, asi como también analizar los errores o riesgos que se cometieron en el mismo,
- Stakeholders (Dueños de RainTamia Plus). – Para la toma de decisiones basada en el control real de existencias.

Referencias y herramientas:

- Estándares: UML 2.5 para el modelado.
- Herramientas:
    - Visual Studio Code como IDE.
    - Mermaid, StarUML como entornos para modelado.
    - ChatGPT, NotebookLM, Gemini, Claude Sonnet como herramientas de IA.
    - Github para control de versiones.

2. Glosario de Términos.

Término	Definición
CIM	*Computation Independent Model*. Modelo que describe el negocio sin tecnología (Requisitos).
PIM	*Platform Independent Model*. Diseño lógico que explica el negocio sin llegar a especficiar las tencologias que se van a utilizar.
PSM	*Platform Specific Model*. Modelo detallado con anotaciones de las tecnologías que se van a utiliza.
Materia Prima	Elementos básicos para la producción (envases de 600ml, tapas, sellos, etiquetas).
Productos Terminados	Corresponde a un elemento que está listo para vender (ej: paca de 20 unidades x 600ml).
Punto de Reorden	Nivel mínimo de stock que dispara automáticamente una solicitud de compra.
Ítem	Termino para agrupar Materia Prima o Productos Terminados.

3. Resultados.

- ## Modelo CIM (Computational Independent Model)

Requisitos Funcionales.

N°	Responsable	RF	Descripción	Priorización (MoSCoW)
RF1	Propietario	Autorización de compras estratégicas	El sistema debe permitir aprobar o rechazar solicitudes de orden de compra de materia prima generadas por el área de gestión de inventario cuando los niveles estén por debajo del mínimo establecido.	S – Should Have
RF2	Contador	Valoración del inventario	El sistema debe permitir la valoración monetaria del inventario de materia prima y productos terminados.	C – Could Have
RF3	Contador	Control de gastos por insumo	El sistema debe registrar el costo unitario de cada materia prima y generar reportes de costos totales.	C – Could Have
RF4	Vendedor	Registro de salidas por venta	El sistema debe descontar automáticamente del inventario los productos terminados asociados a una venta confirmada, tomando en cuenta la cantidad de unidades agrupadas y	M – Must Have

			realizando la conversión automática a unidades base.	
RF5	Encargado de Inventario/Propietario	Alertas de bajo stock y de movimientos realizados.	El sistema debe notificar al encargado de inventario y al propietario cuando los niveles de productos terminados o de materia prima alcancen o caigan por debajo de su punto de reabastecimiento configurado (ej. mínimo de 10 pacas de 20 x 600ml, 500 envases de 600ml)	M – Must Have
RF6	Encargado de inventario	Registro de movimientos de inventario	El sistema debe mantener un historial detallado de movimientos (entradas, salidas, ajustes, daño) con fecha, responsable y motivo, incluyendo movimientos del tipo "daño" o "pérdida" para mercancía dañada o extraviada	M – Must Have
RF7	Encargado de Inventario	Gestión de mínimos y máximos	El sistema debe permitir configurar niveles mínimos y máximos de stock para cada materia prima y producto terminado para activar alertas de reabastecimiento o exceso de inventario.	M – Must Have
RF8	Encargado de Inventario / Vendedor	Gestión de unidades agrupadas	El sistema debe permitir definir, modificar y eliminar reglas de agrupación (por ejemplo, paca de 20 x 600ml).	M – Must Have
RF9	Encargado de Inventario	Generación de solicitud de orden de compra	El sistema debe generar automáticamente una solicitud de orden de compra para los insumos cuyo stock haya caído por debajo del mínimo	M – Must Have

			configurado, debe permitir revisar las cantidades propuestas y enviar la solicitud para su aprobación.	
RF10	Encargado de Inventario	Guardar estado de inventario en un momento determinado	El sistema debe permitir guardar el estado del inventario en un momento determinado, guardando las existencias físicas de todas las materias primas y productos.	M – Must Have
RF11	Encargado de Inventario	Recuperación del estado de inventario.	El sistema debe permitir recuperar el estado de inventario guardado en un momento determinado, permitiendo volver a estados guardados temporalmente.	M – Must Have
RF12	Encargado de Inventario	Gestión de materia prima y productos	El sistema debe permitir la gestión (crear, modificar, eliminar) de la materia prima y productos, junto con sus atributos (nombre, descripción, stock actual, stock mínimo), costo unitario (para materia prima) y precio unitario (para productos).	M – Must Have

Tabla 2. Requisitos Funcionales.

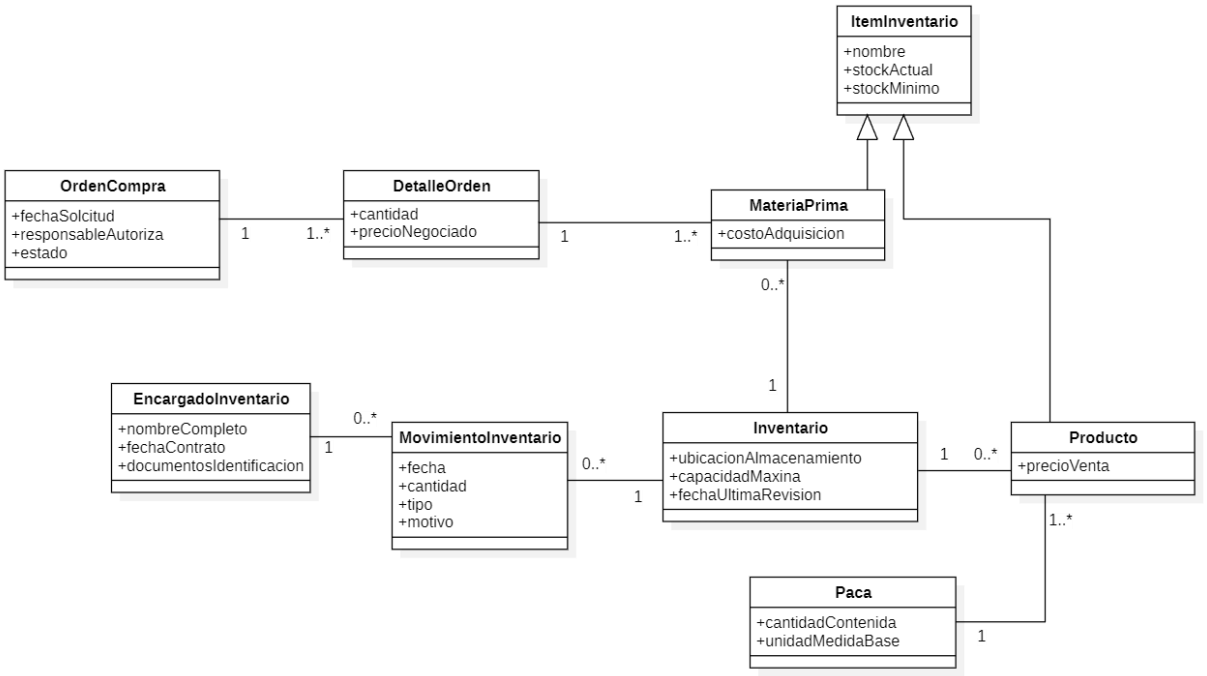
Requisitos No Funcionales.

N°	Tipo	Descripción
RNF01	Rendimiento	La consulta del stock y el registro de movimientos se debe completar en menos de 5 segundos para evitar retrasos en el despacho.
RNF02	Seguridad	El sistema debe restringir la ejecución de movimientos de inventario de tipo Ajuste y Daño/Pérdida exclusivamente al personal que posea el rol de inventario.
RNF03	Trazabilidad	Cada movimiento de inventario registrado debe capturar de manera inmutable la fecha, hora e identificador único del usuario que lo ejecutó, garantizando la auditoría contable.
RNF04	Usabilidad	La interfaz para el registro de movimientos debe ser intuitiva y navegable, permitiendo al encargado de inventario operar el

		sistema con una capacitación que no supere los 30 minutos.
RNF05	Compatibilidad	El módulo de inventario debe ser completamente funcional y con una presentación consistente en los navegadores web Google Chrome y Mozilla Firefox.

Tabla 3. Requisitos No Funcionales.

Modelo Conceptual.



Modelo de Casos de uso

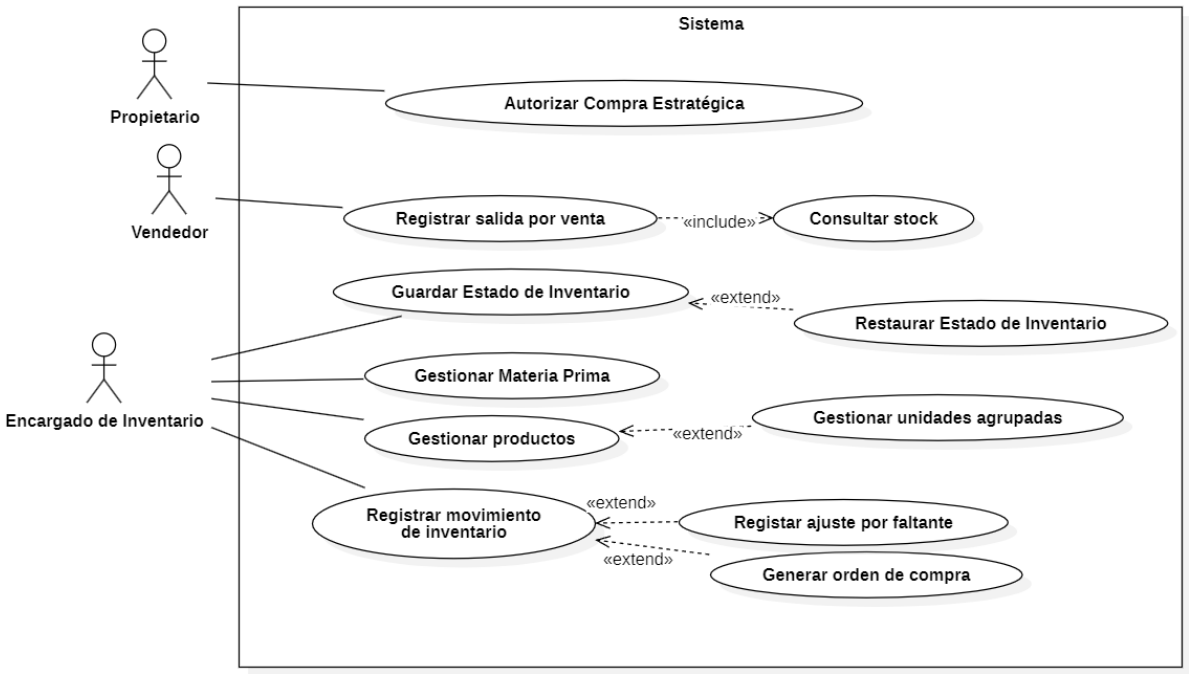


Fig 2. Diagrama de Casos de Uso.

Descripción de Casos de uso.

Nombre	Registrar movimiento de inventario.
Actor Principal	Encargado de Inventario
Objetivo	Permitir el registro formal y detallado de cualquier cambio en el stock físico que no sea una compra o venta (daño, perdida, corrección de conteo), asegurando la trazabilidad.
Precondición	1. El actor debe estar autenticado con el rol "Encargado de Inventario". 2. El ítem (materia prima o producto terminado) debe existir en el catálogo.
Postcondición	El stock del ítem afectado se actualiza. Se genera un nuevo registro en la tabla MovimientoInventario con el tipo "Entrada", "Salida", "Ajuste", "Perdida". Se genera una notificación del movimiento realizado, en caso de que el movimiento resulte en que el stock actual del producto o materia prima este por debajo del mínimo establecido, se generara una notificación adicional sobre el nivel de stock por debajo del mínimo.
Flujo Básico	1. El encargado de inventario accede a la interfaz de Registro Ajuste Movimiento. 2. El sistema muestra una tabla con los productos, la materia prima , y un formulario con el tipo de movimiento (Entrada, Salida, Ajuste, Perdida), cantidad y motivo. 3. El encargado de inventario selecciona un producto o materia prima, el tipo de movimiento, la cantidad a ajustar y un motivo detallado (ejemplo "Faltante en inventario físico", "Botellas rotas en línea de producción). 4. El sistema verifica que se haya seleccionado un producto o materia prima, que la cantidad sea numérica y el motivo este presente. 5. El sistema actualiza la cantidad en el stock, guarda el movimiento con fecha y responsable, y genera una notificación del movimiento realizado. 6. El sistema muestra una confirmación de la operación.
Flujo alternativo.	Flujo alternativo A1 (A partir del Paso 4): El stock queda negativo (solo para ajuste/perdida) A1.1 El sistema calcula que el stock resultante será negativo. A.1.2 El sistema advierte: "La pérdida excede el stock actual. ¿Desea continuar?", si se cancela, el usuario regresa al paso 3. Flujo alternativo A2 (A partir del Paso 5): El stock resultante queda por debajo del stock mínimo. A2.1 El sistema presenta una confirmación de la operación, junto con un

	mensaje en el cual se muestra que el movimiento resulto en un stock por debajo del mínimo. A2.2 El sistema genera una notificación sobre el nivel de stock por debajo del nivel establecido.

Tabla 4. Caso de uso detallado "Registrar movimiento de inventario".

Nombre	Gestionar Productos/Materia Prima
Actor Principal	Encargado de Inventario
Objetivo	Permitir la creación, edición o eliminación lógica de los ítems (materia prima/productos).
Precondición	1. El actor debe estar autenticado con el rol "Encargado de Inventario".
Postcondición	El registro de la Materia Prima/Producto se actualiza en la base de datos. Los niveles de stock actual y mínimo quedan vinculados al nuevo ítem.
Flujo Básico	1. El encargado de inventario accede a la interfaz correspondiente (Gestionar Productos Terminados/ Gestionar Materias Primas). 2. El sistema muestra una lista de los ítems ya existentes, y un formulario para el registro de un nuevo ítem. 3. El encargado de inventario ingresa los datos requeridos (nombre, descripción, stock actual, stock mínimo), costo unitario (para materia prima) y precio unitario (para productos). 4. El sistema valida que el campo de nombre este completo como mínimo. 5. El sistema guarda la información y confirma el éxito de la operación.
Flujo alternativo.	Flujo alternativo A1 (A partir del Paso 1): Edición de Ítem A1.1 El encargado de inventario selecciona un ítem el cual quiera editar. A1.2 El sistema carga los datos del ítem seleccionado en el formulario para su edición. A1.3 El encargado modifica los campos necesarios, y pulsa modificar. A1.4 El sistema actualiza la información del ítem modificado. Flujo alternativo A2 (A partir del Paso 1): Borrado de ítem A2.1 El encargado de inventario selecciona la opción de borrar del ítem. A2.2 El sistema muestra un mensaje de confirmación para eliminar el ítem. A2.3 El encargado confirma la acción. A2.4 El sistema elimina la información del ítem.

Nombre	Guardar Estado de Inventario
Actor Principal	Encargado de Inventario
Objetivo	Capturar el estado del inventario (información del inventario, stock de todos los ítems) de manera temporal en un momento determinado.
Precondición	1. El actor debe estar autenticado con el rol "Encargado de

	Inventario".
Postcondición	1. Se genera un registro temporal del estado del inventario. 2. Se almacenan las cantidades exactas de ítems al momento de guardar el estado del inventario.
Flujo Básico	1. El encargado de inventario accede a la interfaz Estado del Inventario. 2. El sistema muestra un resumen previo del estado actual del inventario, cantidad de tipos de materia prima, de productos, y un total de ambos, además del stock actual con el que cuenta cada uno de estos, y un total general. 3. El encargado de inventario toca el botón de Guardar Snapshot 4. El sistema procesa la información y guarda el estado actual del inventario de manera temporal. 5. El sistema muestra un mensaje de "Estado del inventario guardado exitosamente".
Flujo alternativo.	

Nombre	Restaurar Estado de Inventario
Actor Principal	Encargado de Inventario
Objetivo	Recuperar información de stock de un estado temporal guardado previo para revertir el estado actual en caso de errores masivos de registro.
Precondición	1. El actor debe estar autenticado con el rol "Encargado de Inventario". 2. Debe haber un estado de inventario guardado temporalmente.
Postcondición	1. Se sobreescribe el inventario actual con los valores del estado del inventario guardado.
Flujo Básico	1. El encargado de inventario accede a la interfaz Estado del Inventario. 2. El sistema muestra un resumen previo del estado actual del inventario, cantidad de tipos de materia prima, de productos, y un total de ambos, además del stock actual con el que cuenta cada uno de estos, y un total general. 3. El encargado de inventario toca el botón de Restaurar Estado Anterior 4. El sistema muestra un mensaje para asegurar la acción. 5. El encargado de inventario confirma la acción. 6. El sistema sobrescribe los datos actuales del inventario por los datos del estado del inventario temporal. 7. El sistema muestra un mensaje de "Estado del inventario restaurado exitosamente"

Flujo alternativo.	Flujo alternativo A1 (A partir del Paso 5): No hay estados anteriores A1.1 El sistema no encuentra datos de un estado de inventario guardado. A1.2 El sistema muestra un mensaje de “No hay estados anteriores para restaurar”.

Nombre	Generar Orden de Compra
Actor Principal	Encargado de Inventario/ Sistema (Iniciador)
Objetivo	Automatizar la detección de la necesidad de reabastecimiento para la materia prima, generando una solicitud formal para la aprobación gerencial.
Precondición	2. El actor debe estar autenticado con el rol "Encargado de Inventario" para revisar o enviar la solicitud. 3. El stock de una o más Materias Primas (ej. tapas, envases, etiquetas) ha caído por debajo del nivel mínimo establecido (RF7).
Postcondición	3. Se crea un nuevo registro de SolicitudCompra con estado "PENDIENTE DE APROBACIÓN". 4. Se notifica al Propietario (mediante el Servicio de Aprobación Gerencial) para su revisión.
Flujo Básico	8. El sistema (a través de un trigger) realiza la validación de los niveles de stock. 9. El sistema identifica que el stock de X ítem (por ejemplo, envases de 600ml) ha caído por debajo del nivel mínimo establecido. 10. El sistema crea una nueva solicitud de orden de compra que incluye al ítem y la cantidad sugerida (por ejemplo, podría establecerse hasta el nivel máximo de dicho ítem). 11. El encargado de inventario recibe una alerta para revisar la Solicitud generada (opcionalmente puede ajustar la cantidad según vea necesario). 12. El encargado de inventario envía la solicitud (el sistema también puede enviarla) al servicio externo para la autorización de la compra. 13. El sistema notifica al Propietario que tiene una solicitud pendiente de aprobación. 14. El proceso pasa a un estado de espera hasta que el gerente general interactúe con el sistema externo.
Flujo alternativo.	Flujo alternativo A1: Stock se corrigió antes de enviar la solicitud. A1.1 El encargado de inventario registra una entrada o un ajuste positivo justo después de que el sistema detecto el nivel bajo de stock, pero antes de enviar la solicitud.

	A1.2 El sistema valida que el stock del ítem X ya ha superado el nivel mínimo establecido. A.1.3 El sistema cancela el envió y notifica al encargado de inventario que "El reabastecimiento ya no es necesario". Flujo alternativo A2: Falla en el servicio de aprobación. A2.1 El sistema intenta conectarse con el servicio de aprobación gerencial y falla. A.2.2 El sistema almacena la solicitud en un estado de "FALLA DE ENVIO" y notifica al encargado de inventario para que la reintente manualmente.

Tabla 5. Caso de uso detallado "Registrar movimiento de inventario".

Actores Involucrados.

Actor	Funcionalidades Clave
Usuarios Finales	
Encargado de Inventario	• Encargado de la ejecución física y el registro de entradas/salidas/ajustes. • Configuración de mínimos/máximos y unidades agrupadas. • Generación de solicitud de orden de compra. • Recepción de alertas de bajo stock.
Vendedor	• Iniciador de la salida por venta. • Consulta de stock disponible.
Administradores	
Contador	• Valoración del inventario. • Control del costo unitario
Propietario	• Aprobación de compras.
Sistemas Externos	
Módulo de Ventas	• Interactúa con el módulo a través del registro por salida de venta.
Módulo de Contabilidad	• Interactúa con el módulo para obtener datos financieros
Servicio de Aprobación Gerencial	• Interactúa con el módulo para la aprobación de órdenes de compra.

- ## Modelo PIM (Platform Independent Model)
  - Diagrama de clases gdel dominio completo

Modelo de Dominio

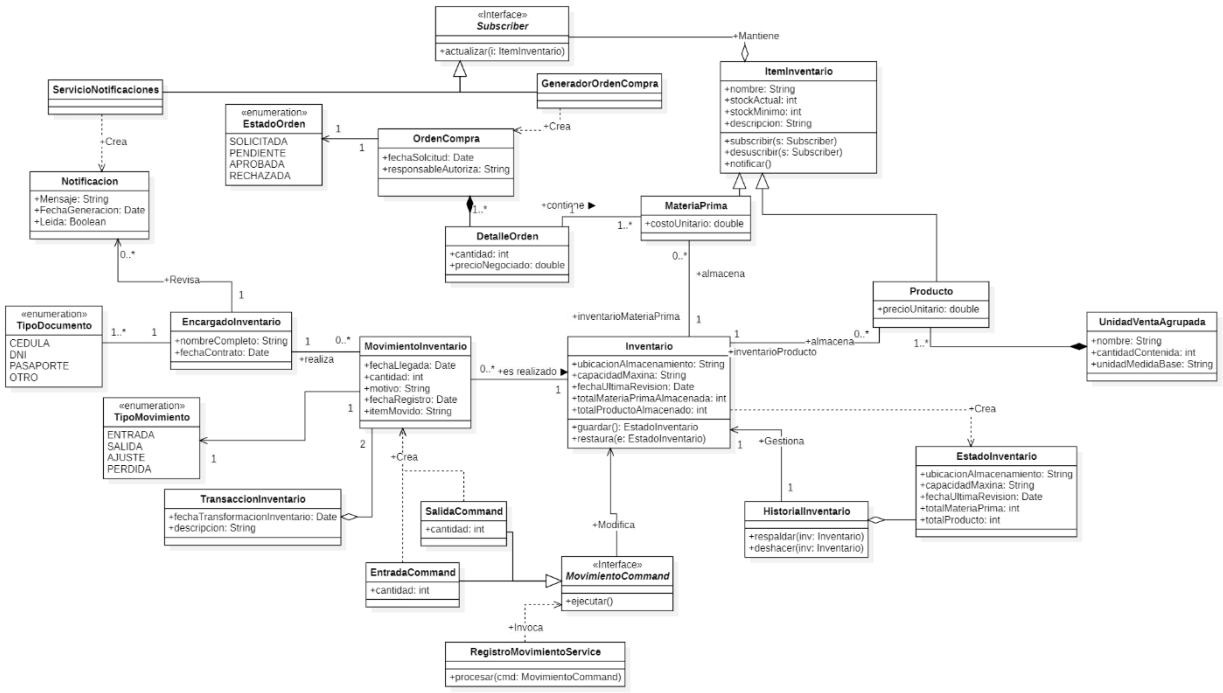


Diagrama de Paquetes

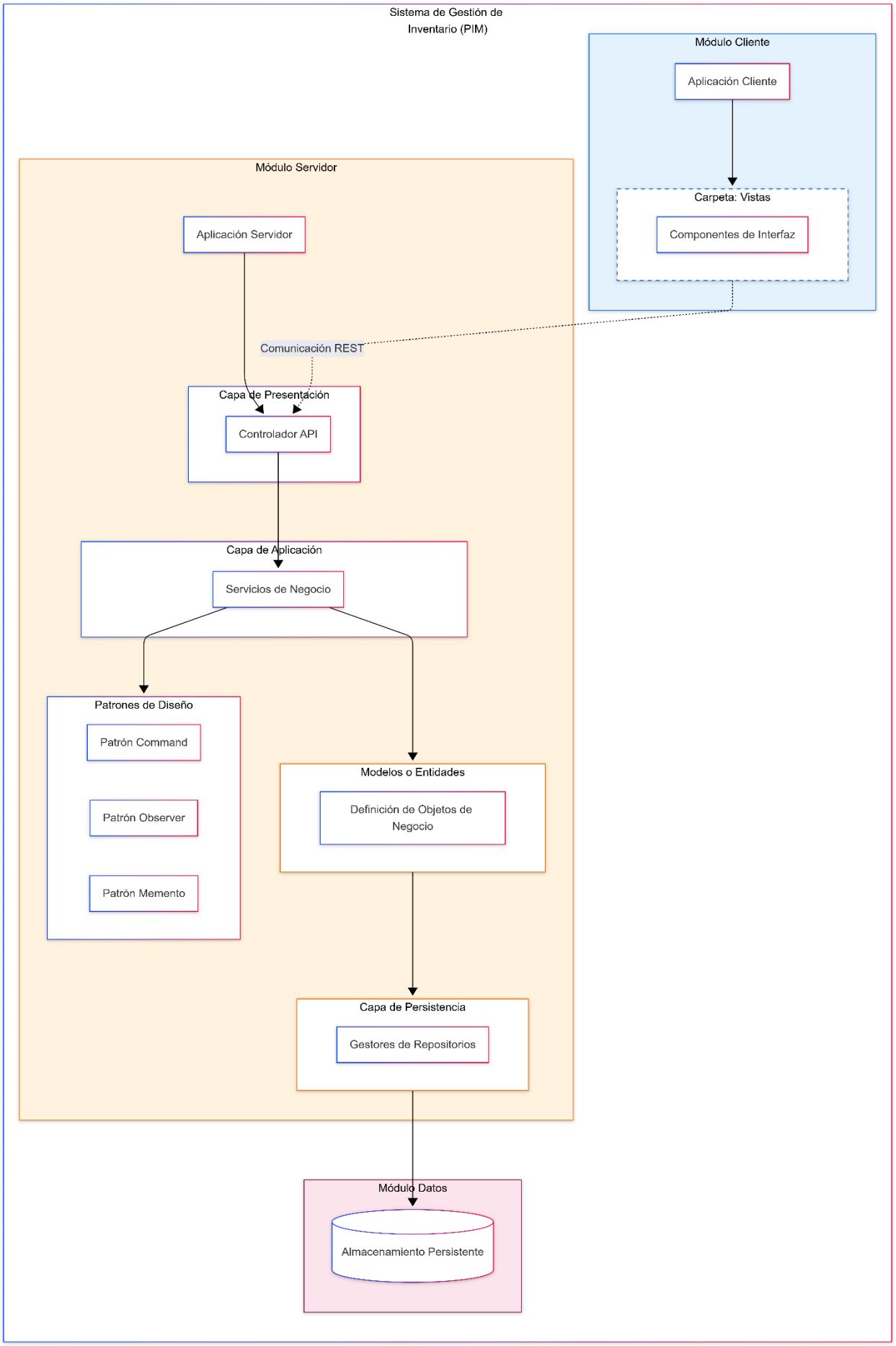
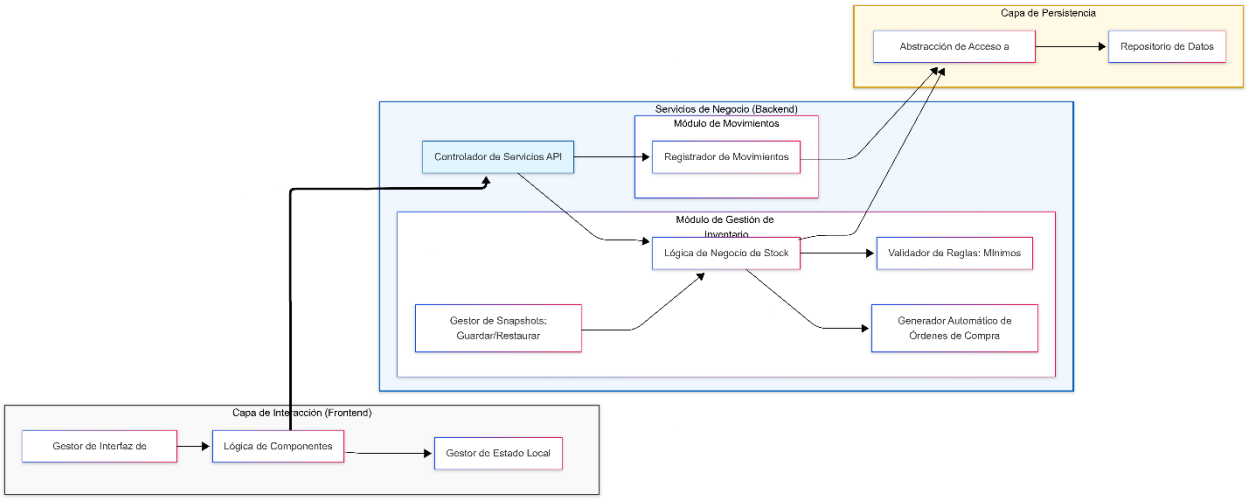


Diagrama de Componentes.



Decisiones de Diseño

- **Patrones Aplicados**

Patrón	Descripción	Justificación
Account	Usado para gestionar valores que cambian en el tiempo y requieren trazabilidad.	Elegido para mejorar la precisión temporal, auditabilidad y trazabilidad de la evolución de un valor a lo largo del tiempo
Summary Account	Usado para la organización de cuentas con una estructura jerárquica, usando una cuenta de alto nivel como contenedor lógico.	Seleccionado para facilitar la consolidación de multiples cuentas (materia prima y productos en este caso)
MVC	Aplicado para separar la logica de datos del control	Permite delegar la responsabilidad de la interfaz visual al frontend, manteniendo al backend enfocado en la lógica.
Repository	Actua como un mediador entre la logica de negocio y la persistencia de datos.	Encapsula la lógica de acceso a datos, haciendo que la capa de negocios sea agnóstica al motor de base de datos.
API Gateway	Centralización de peticiones en el cliente del frontend hacia los recursos del backend.	Maneja de forma unificada las peticiones y errores, ocultando la complejidad de los servicios internos al usuario final.
Memento	Usado para capturar y restaurar estados del inventario.	Necesidad de registrar y restaurar estados temporales del inventario en caso de errores cometidos en el registro de entradas o salidas.

Observer	Aplicado para el sistema de notificaciones.	Seleccionado para permitir que el inventario notifique automáticamente obre cambios de stock.
Command	Usado para encapsular operaciones de entrada y salida de inventario como objetos independientes.	Elegido para tratar cada movimiento como un objeto único.

- **Estilos Arquitectónicos**

Estilo Arquitectónico	Descripción	Justificación
Arquitectura en Capas	El sistema se organiza en 3 capas: De Presentación (Frontend), de Lógica (Backend) y de Persistencia (Datos)	Seleccionada para separación de responsabilidades entre capas.
Cliente-Servidor	Esquema en el cual existe un proveedor, o servidor, el cual debe atender a las peticiones de un demandante o también llamado cliente.	Proporciona una comunicación jerárquica y controlada, permitiendo que el servidor orqueste las reglas de negocio de manera centralizada.

- ## PSM (Platform Specific Model)

Tecnologías Conretas.

Componente	**Tecnologia**	**Version**
Lenguaje Backend	Python	3.13.5
Framework Backend	Django	6.0
Framework Frontend	Svelte	3.59.2
Base de datos	SQLite	3.49.1

Tecnologías Conretas.

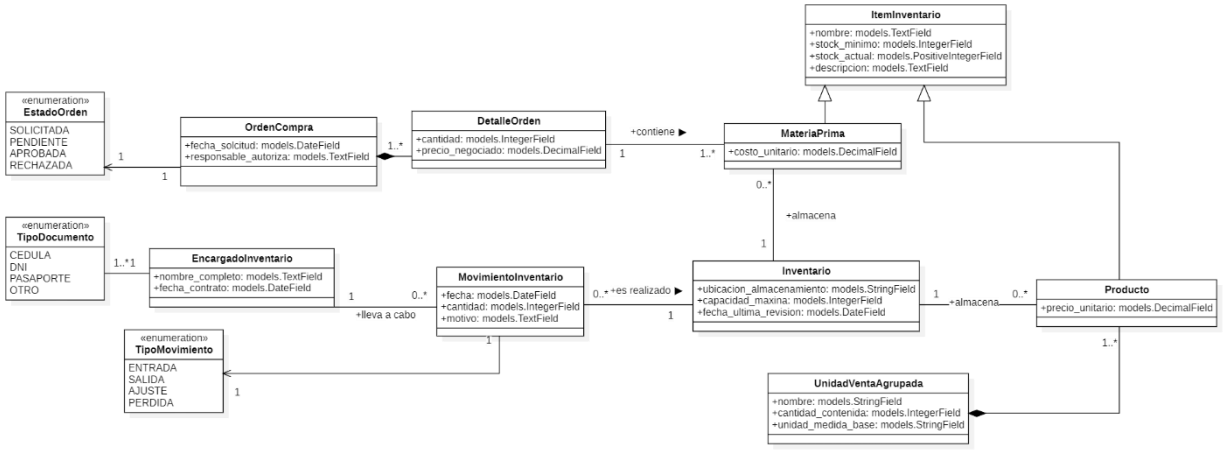
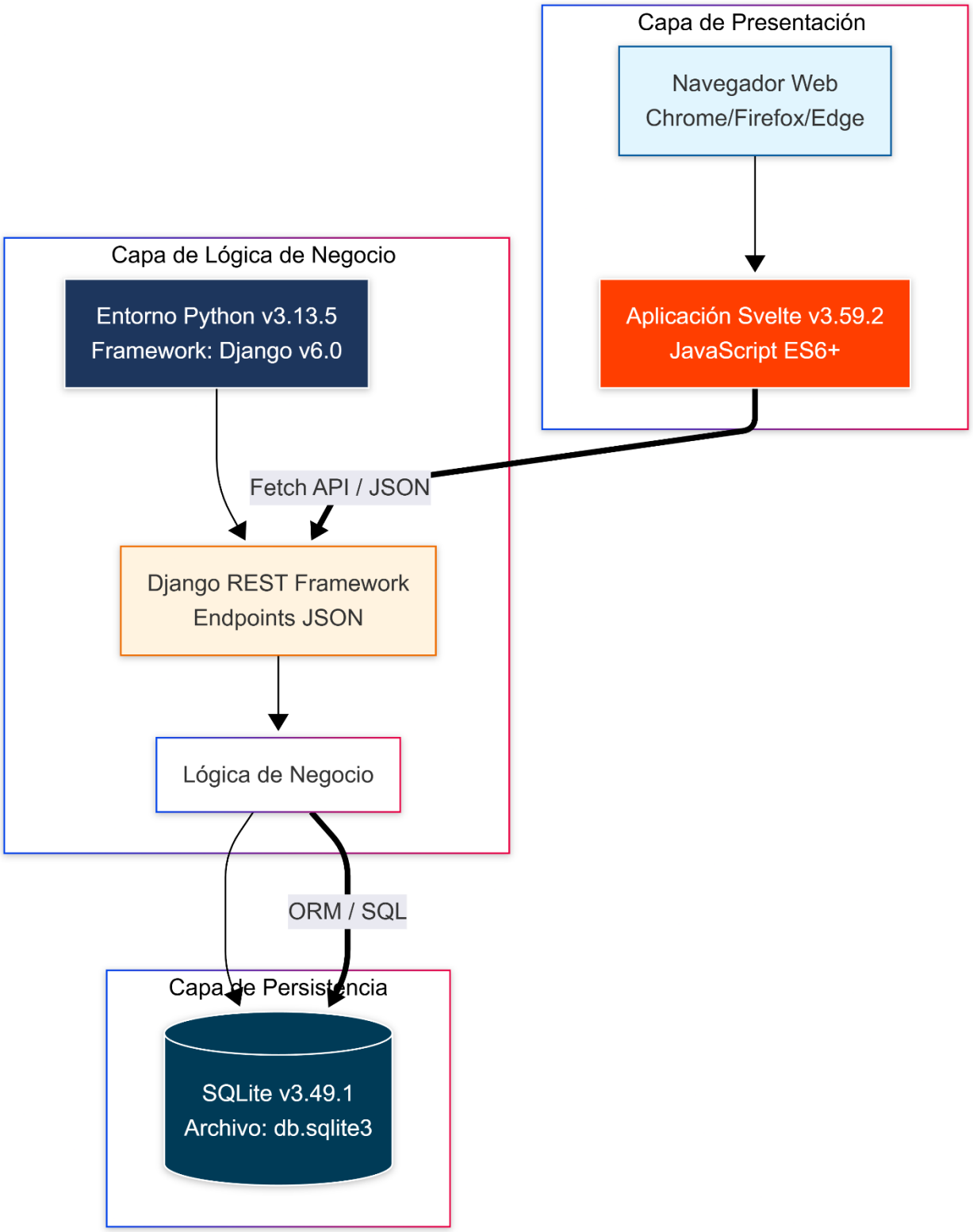


Diagrama de despliegue



Transformaciones MDA

Herramienta usada para transformación:

- StarUML: Actuando como entorno de modelado.
- Extensión Django para StarUML: Se uso este motor de transformación apra automatizar la generación de archivos para models.py y a partir del diagrama de clases, reduciendo errores manuales y asegurando que la arquitectura del software sea el fiel reflejo del modelo de negocio.}

Reglas usadas para transformaciones:

- Los atributos de las clases se transformaron en campos específicos usados por Django,

como models.CharField, models.DateField, etc.

- ## Modelo IM (Implementation Model)

Transformacion de PSM a Codigo:

Para la generación del código base, se uso la extensión de Django para StarUML, este proceso permitió:

- Mapeo de atributos a campos específicos que usa Django como models.CharField, models.StringField, etc.
- Garantizar que la estructura de clases definida en el diseño se replique exactamente en la estructura de archivos del Backend.

En cuanto a la distribución del esfuerzo de codificación, se desglosa de la siguiente manera:

- Generacion automática de 20%, lo que corresponde a la definición de la capa de datos (Modelos), StarUML gneero el esqueleto estructural y las relaciones de base de datos.
- Desarrollo manual de 80%, lo que corresponde a la implementación de Reglas de Negocio, validaciones, logica de patrones de diseño y desarrollo de interfaz en Svelte.

Ejemplos:

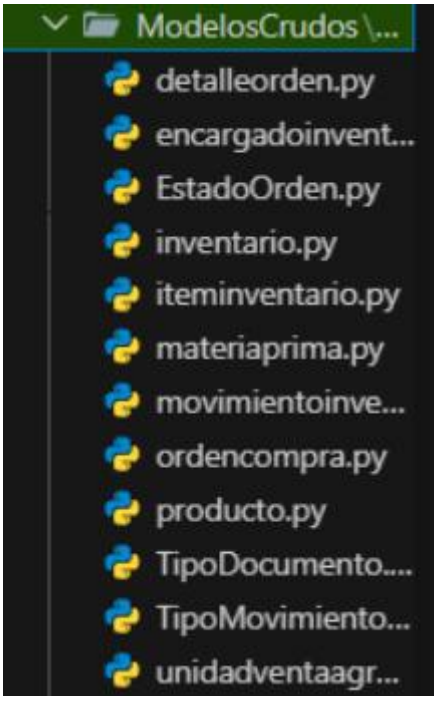


Fig 9. Código generado a partir del modelo PSM

```python
class RegistrarMovimientoView(APIView):
    """
    Vista para registrar movimientos de inventario.
    Implementa el caso de uso "Registrar movimiento de inventario"
    """

    def post(self, request):
        serializer = RegistrarMovimientoSerializer(data=request.data)

        if not serializer.is_valid():
            return Response(
                serializer.errors,
                status=status.HTTP_400_BAD_REQUEST
            )

        validated_data = serializer.validated_data
        item = validated_data['item']
        encargado = validated_data['encargado']
        tipo_movimiento = validated_data['tipo_movimiento']
        cantidad = validated_data['cantidad']
        motivo = validated_data['motivo']

        # Verificar advertencia de stock negativo
        advertencia = validated_data.get('advertencia')
        confirmar = request.data.get('confirmar', False)

        if advertencia and not confirmar:
            return Response({
                'advertencia': advertencia,
                'requiere_confirmacion': True
            }, status=status.HTTP_200_OK)

        # Actualizar stock según el tipo de movimiento
        if tipo_movimiento == TipoMovimiento.ENTRADA:
            item.stock_actual += abs(cantidad)
        elif tipo_movimiento in [TipoMovimiento.SALIDA, TipoMovimiento.PERDIDA]:
            item.stock_actual -= abs(cantidad)
        elif tipo_movimiento == TipoMovimiento.AJUSTE:
            # Para ajuste, cantidad positiva suma, negativa resta
            item.stock_actual += cantidad

        item.save()

        # Crear el movimiento
        movimiento = MovimientoInventario.objects.create(
            fecha=timezone.now().date(),
            cantidad=cantidad,
            motivo=motivo,
            tipo_movimiento=tipo_movimiento,
            encargado=encargado
        )

        # Asociar con el inventario del item
        movimiento.inventarios.add(item.inventario)

        # Serializar respuesta
        movimiento_serializer = MovimientoInventarioSerializer(movimiento)

        # Serializar el item actualizado
        if validated_data['item_tipo'] == 'materia_prima':
            item_serializer = MateriaPrimaSerializer(item)
        else:
            item_serializer = ProductoSerializer(item)

        return Response({
            'mensaje': 'Movimiento registrado exitosamente',
            'motivo': motivo,
            'movimiento': movimiento_serializer.data,
            'item_actualizado': item_serializer.data,
            'stock_anterior': item.stock_actual - cantidad if tipo_movimiento == TipoMovimiento.ENTRADA
else item.stock_actual + abs(cantidad),
            'stock_actual': item.stock_actual
        }, status=status.HTTP_201_CREATED)
```

Fig 10. Código implementado de manera manual

- Pruebas unitarias básicas.

Pruebas unitarias en video: [Video Demo]

4. Conclusiones, recomendaciones y aprendizajes.

Conclusiones.

- El flujo de trabajo de MDA permite una transición coherente desde los requisitos que son representados como CIM hasta el código generado conocido como IM. La definición previa de un PIM solido facilito bastante que el cambio hacia el PSM (siendo específicamente para Django) fuera una tarea únicamente de mapeo técnico y no de llevar a cabo tareas de rediseño lógico.
- Gracias a documentar primeramente la lógica de negocio en PUM, el sistema queda preparado para ser portado a otras tecnologías en caso de que sea necesario (como Node.js o Spring boot) en el futuro, requiriendo únicamente que se transforme a nivel de PSM.

Aprendizajes.

- Se comprendió de manera más profunda como en MDD, los modelos realmente llevan el papel como el eje central de todo el proyecto, indicando como se trabajara, como lo es en el diagrama de componentes, el cual sirvió de guía para poder organizar la estructura de carpetas en el backend de Django.

Recomendaciones.

- Para un posible crecimiento en la demanda de la empresa RainTamia Plus, se recomienda transformar el modelo actual que es monolítico, hacia un modelo PSM de microservicios, de este modo, por ejemplo, los módulos de “Inventario” y “Ventas” serán artefactos de despliegue independientes.

5. Declaración de uso de IA

- Se usaron herramientas Gemini (Flash 2.5) para organizar y estructurar las ideas del presente trabajo.
- Se usaron herramientas Claude Sonnet (4.5) como un apoyo para codificación rápida en el presente trabajo.
- Se usaron herramientas NotebookLM para resumir información de artículos para una investigación previa sobre escenarios similares al caso de estudio.

6. Anexos

- Repositorio en Github: Repositorio